

Version 2.0

January 2026



OPERATING SYSTEMS PRACTICUM

MODULE 1 - INTRODUCTION TO OPERATING SYSTEMS AND COMPUTER SYSTEM STRUCTURE

Author:

Ir. Denar Regata Akbi, S.Kom., M.Kom.

Putri Nayla Sabri

Muhammad Zamzam Baihaqi

INFORMATICS LABORATORY

University of Muhammadiyah Malang

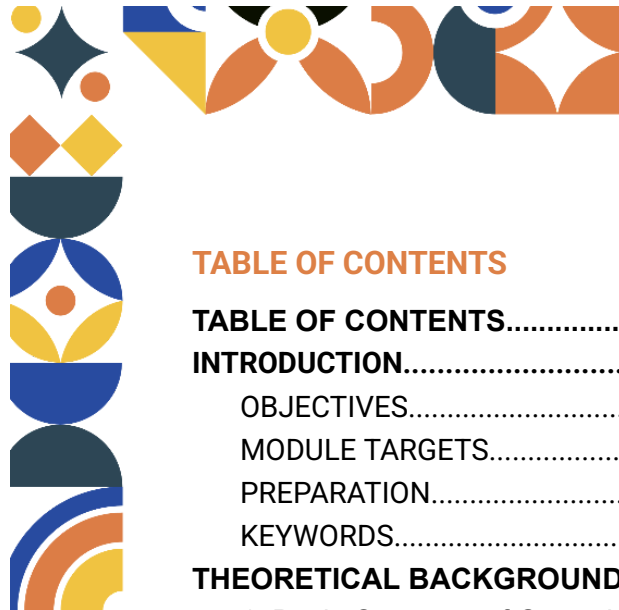
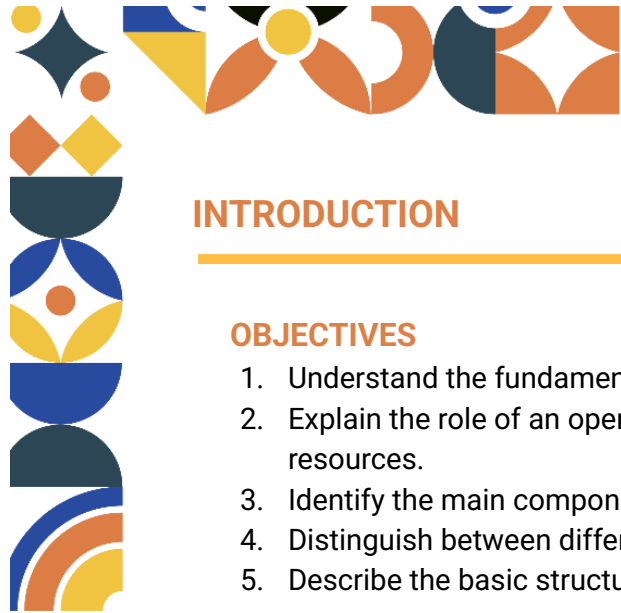


TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
INTRODUCTION.....	3
OBJECTIVES.....	3
MODULE TARGETS.....	3
PREPARATION.....	3
KEYWORDS.....	3
THEORETICAL BACKGROUND.....	4
1. Basic Concepts of Operating Systems.....	4
1.1 Definition of Operating System.....	4
1.2 Main Functions of Operating System.....	4
1.3 Characteristics of Linux as an Operating System.....	6
2. Computer System Structure.....	7
2.1 Hardware Components.....	7
2.2 Role of the Operating System in Hardware Management.....	8
2.3 System Layers.....	10
3. Linux File System and Shell.....	11
3.1 Linux File System Concept.....	11
3.2 Basic Linux File System Hierarchy.....	11
3.3 Shell and Command Line Interface.....	13
4. Basic File System Navigation (CLI).....	13
4.1 Path Concept.....	13
4.2 Basic Navigation Commands.....	14
EXERCISE 1 : The Traveler (Path Concept).....	15
EXERCISE 2 : The Detective (Exploring Output).....	15
EXERCISE 3 : The Architect (File Management).....	15
EXERCISE 4 : The Cleanup (System Maintenance).....	15
EXERCISE 5 : The Root Observer (Nested Folders & System Tree).....	15
4.3 Command Workflow.....	16
PRACTICAL APPLICATIONS.....	17
PRACTICE 1.....	17
PRACTICE 2.....	17
DEMO - CASE STUDY.....	17
ASSESSMENT RUBRIC.....	19
ASSESSMENT SCALE.....	19





INTRODUCTION

OBJECTIVES

1. Understand the fundamental concepts and functions of an operating system.
2. Explain the role of an operating system in managing computer hardware and software resources.
3. Identify the main components of a computer system and their interactions.
4. Distinguish between different types of operating systems.
5. Describe the basic structure and architecture of modern computer systems.

MODULE TARGETS

1. Have a clear understanding of operating system fundamentals.
2. Recognize the relationship between hardware, operating system, and application software.
3. Explain basic computer system structures, including CPU, memory, storage, and I/O devices.
4. Prepare an appropriate environment for operating system practice sessions.
5. Build a foundational knowledge for advanced operating system topics in subsequent modules.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson
رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا
2. Laptop or personal computer
3. Rocky Linux
4. Virtual Box
5. Strong motivation and commitment to learning

KEYWORDS

Operating systems, Computer System Structure, System architecture





THEORETICAL BACKGROUND

1. Basic Concepts of Operating Systems

1.1 Definition of Operating System

An **Operating System (OS)** is the core software that functions as an intermediary between users, applications, and computer hardware. Without an operating system, a computer would merely be a collection of physical components that cannot be utilized effectively.

In general, the OS is responsible for managing system resources such as the processor, memory, storage, and input/output devices. Applications executed by users do not interact directly with the hardware; instead, they rely on the services provided by the operating system.

In the Linux approach, an operating system is understood as a combination of the **kernel** and a set of **user utilities**.

- The kernel is responsible for managing direct interactions with the hardware
- The user utilities provide interfaces and tools for users and applications.

Although this approach is commonly associated with Linux, the concept of separating the system core from supporting components is also found in other operating systems such as Windows and macOS, albeit with different architectures and terminologies.

From a functional perspective, an operating system can be analogized as a **traffic manager** within a computer system. The OS determines:

- which processes are allowed to use the CPU and when they are scheduled
- how much memory can be allocated to each application
- when data is stored to or retrieved from secondary storage
- which devices are active or waiting for access.

By managing these aspects, the operating system ensures that system resources are utilized efficiently, in a controlled manner, and fairly among applications.

1.2 Main Functions of Operating System

A. Process Management

Process management deals with the control and coordination of programs that are currently being executed. The operating system is responsible for:

- Creating and terminating processes
- Scheduling processes so that the CPU can be shared among them
- Managing process states (such as running, waiting, and terminated)



Without proper process management, a system may experience conditions such as **system hangs** or **starvation**, where a single process monopolizes the CPU for an extended period of time.

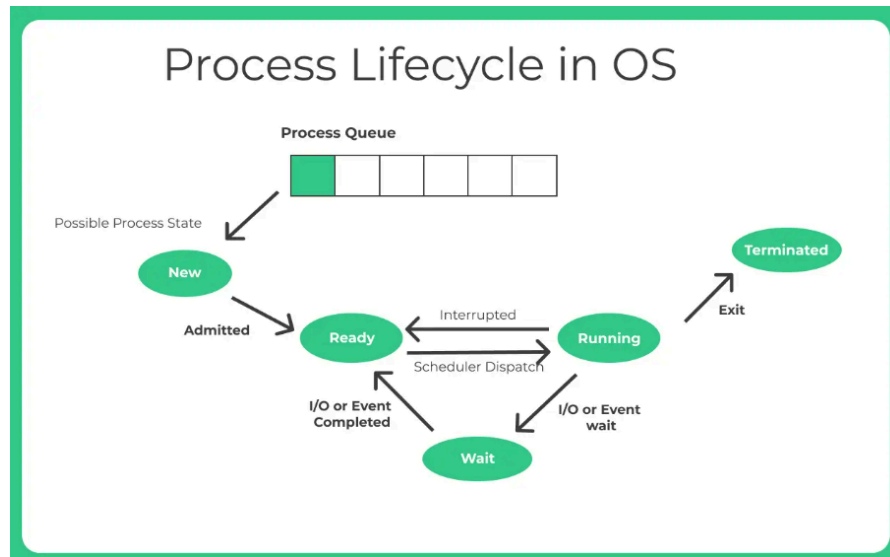


Fig 1. Process Life Cycle Diagram

B. Memory Management

Memory management is responsible for ensuring that each application receives sufficient memory space without interfering with other processes. The operating system manages:

- Memory allocation and deallocation
- Memory protection between processes
- The use of virtual memory when physical memory is limited

This concept is crucial for maintaining system stability, especially when multiple applications are running simultaneously.

C. Storage Management and File System

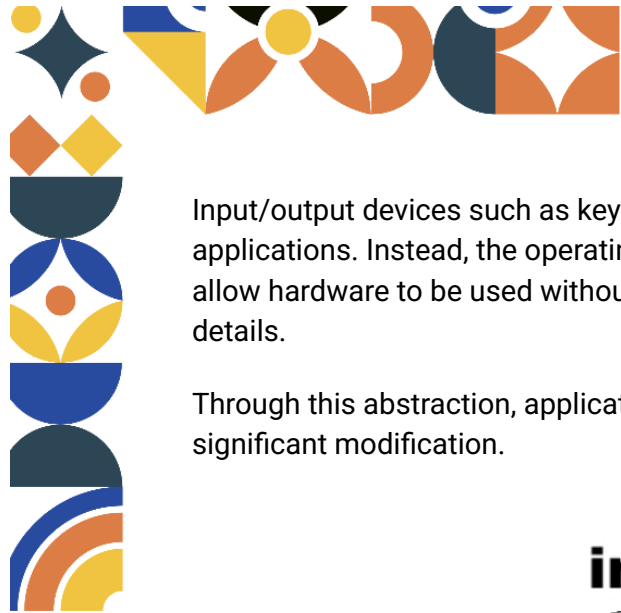
The operating system manages how data is stored and organized on storage media. This function includes:

- Organizing file and directory structures
- Managing free and used storage space
- Controlling access permissions to files

Although different operating systems use different file systems, they share the same objective: to store data in a structured, secure, and reliable manner.

D. Device and I/O Management





Input/output devices such as keyboards, mice, printers, and disks are not accessed directly by applications. Instead, the operating system provides **device drivers** and **I/O mechanisms** that allow hardware to be used without requiring applications to understand hardware-specific details.

Through this abstraction, applications can run on different hardware platforms without significant modification.

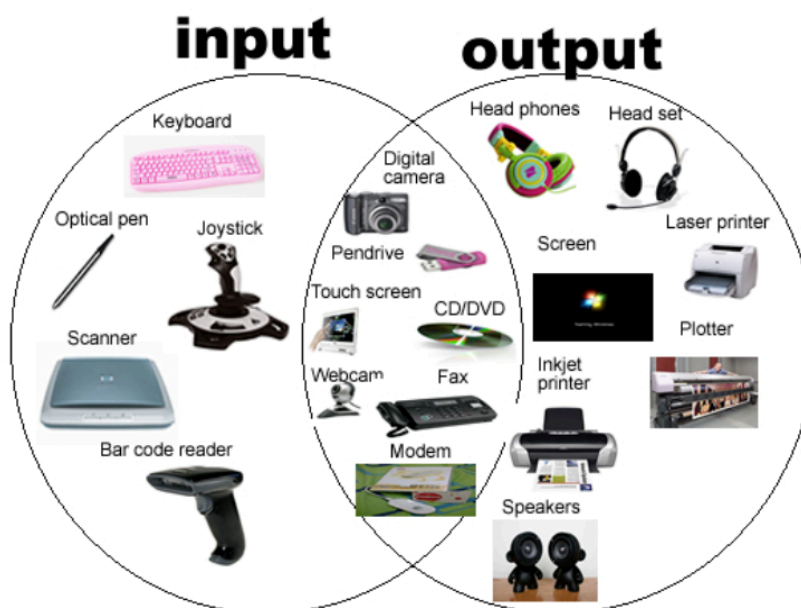


Fig 2. I/O Devices

E. Security and Access Control

The operating system is also responsible for protecting the system from unauthorized access. This is achieved through:

- User authentication mechanisms
- File and system permission management
- Process isolation to prevent interference between applications

Operating system security is particularly important in multi-user environments and network-connected systems.

1.3 Characteristics of Linux as an Operating System

Linux is one of the most widely used operating systems, particularly in server environments and software development contexts. Some of its main characteristics include:





a. Multi-user

Linux supports multiple users operating on the system simultaneously. Each user has their own access rights and working environment, making Linux suitable and secure for shared systems.

b. Multi-tasking

Linux is capable of executing multiple processes concurrently. The operating system manages CPU time allocation efficiently, allowing users to run several applications without significantly affecting overall system performance.

c. Stability and Security

Linux is well known for its high level of stability. System crashes are relatively rare due to strong process isolation, strict memory management, and permission-based security model. These characteristics are key reasons why Linux is commonly used in servers and mission-critical systems.

d. Modular Architecture (Kernel and User Utilities)

Linux employs a modular architecture in which the kernel can be extended through loadable modules, while user utilities can be developed independently. This approach makes the system flexible, easy to maintain and extend, and efficient and adaptable to user requirements.

2. Computer System Structure

The computer system structure explains how **hardware and software components interact** to perform computational tasks. Understanding this structure is essential to view the role of the **Operating System (OS)** comprehensively, not merely as software, but as the overall system coordinator.

2.1 Hardware Components

a. Central Processing Unit (CPU)

The Central Processing Unit (CPU) is the core of a computer system responsible for executing program instructions. It performs fundamental operations such as arithmetic calculations, logical operations, and control of program execution flow.

Although the CPU operates at very high speed, it cannot function independently. However, it requires an operating system to determine which processes are executed, manage context switching between programs, and optimize the utilization of processing time.



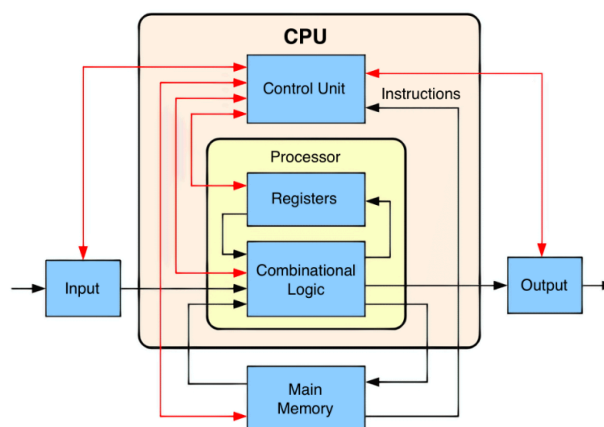
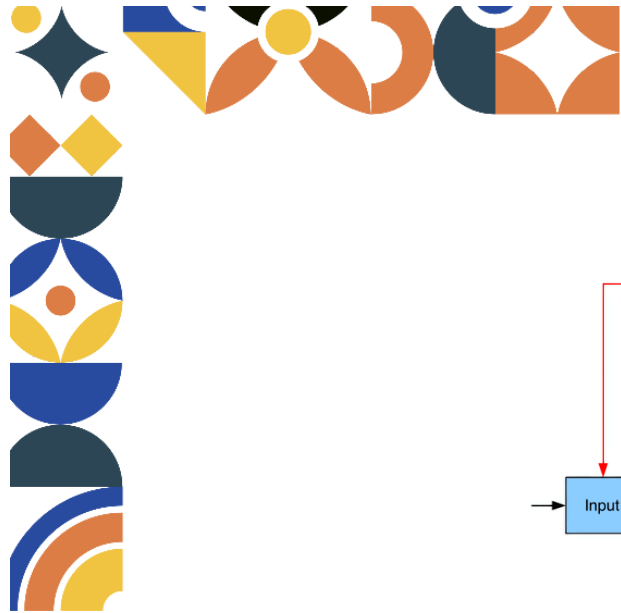


Fig 3. CPU Components Diagram

b. Main Memory (RAM)

Main memory, commonly known as RAM, serves as temporary storage for programs that are currently running and data that is actively being processed. RAM is **volatile**, meaning that its contents are lost when the computer is powered off. Due to its limited capacity, effective memory management is required to allow multiple applications to run simultaneously without conflict.

c. Storage (Disk)

Storage is used for **long-term data storage**, including the operating system, applications, and user files. Common storage media include hard disk drives (HDDs) and solid-state drives (SSDs). Unlike RAM, storage is **non-volatile**, so data remains available even when the system is turned off. The operating system plays a critical role in organizing, retrieving, and protecting data stored on these devices.

d. Input and Output (I/O) Devices

Input and output devices enable interaction between users and the computer system. Input devices such as keyboards, mice, and scanners allow users to provide data, while output devices like monitors, printers, and speakers present processed information. Because each device has different characteristics, the operating system is responsible for managing them uniformly through specialized I/O mechanisms.

2.2 Role of the Operating System in Hardware Management

The operating system functions as the **main controller of hardware interactions**, allowing applications to operate without direct access to physical hardware components.

a. CPU Scheduling

Since a CPU can execute only one instruction at a time, the operating system employs **scheduling techniques** to distribute CPU time among multiple processes. Scheduling ensures



that each process receives execution time, the system remains responsive, and no single process monopolizes the CPU for an excessive duration.

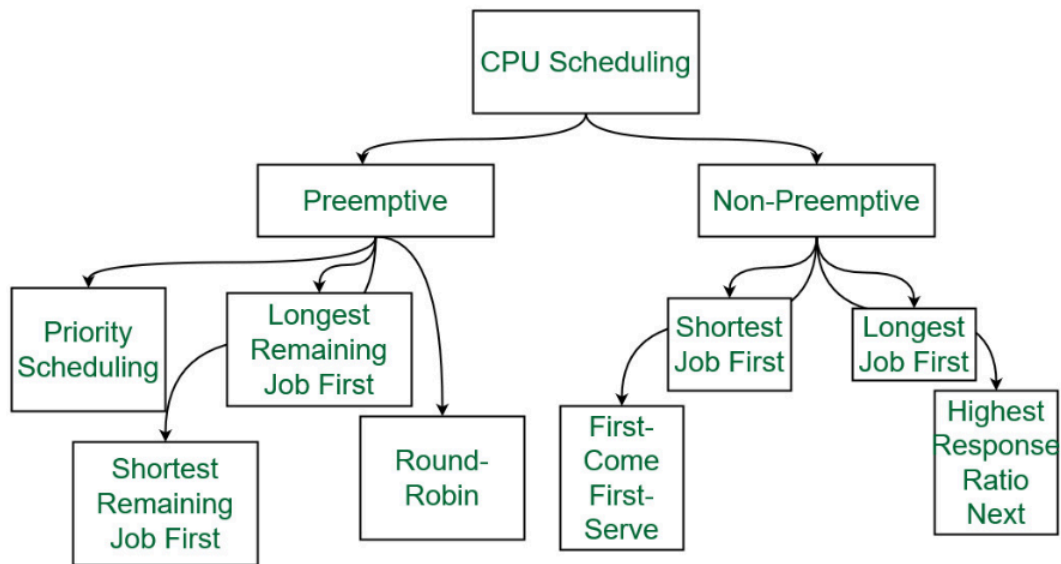


Fig 4. Process Scheduling Diagram

b. Memory Isolation and Protection

The operating system ensures that each process accesses only its own allocated memory space and cannot read or modify the memory of other processes. This memory isolation is essential to prevent errors in one application from disrupting the entire system.

c. Disk Access through the File System

Applications do not interact directly with storage devices. Instead, the operating system provides a **file system** that acts as an intermediary for reading and writing data, organizing directory structures, and controlling file access permissions. Through this abstraction, the physical details of storage devices are hidden from users and applications.





```
praktikumso@localhost:~$ tree -L 1 /
/
├── afs
├── bin -> usr/bin
├── boot
├── dev
├── etc
├── home
├── lib -> usr/lib
├── lib64 -> usr/lib64
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin -> usr/sbin
├── srv
├── sys
├── tmp
├── usr
└── var

21 directories, 0 files
```

Fig 5. File System Structure

d. Device Management through Drivers and Interrupts

The operating system communicates with hardware components using **device drivers**, which serve as translators between the OS and physical devices. In addition, hardware devices generate **interrupts** to notify the CPU when certain events occur, such as keyboard input or the completion of an I/O operation. This mechanism improves efficiency by preventing the CPU from continuously checking device status.

2.3 System Layers

A computer system can be viewed as a **layered structure**, where each layer has a specific role and responsibility.

a. User

The user represents the entity that interacts with the system, either directly or through applications. Users do not communicate with hardware directly but through intermediary software layers.

b. Application

Applications are software programs designed to meet user needs, such as word processors, web browsers, or data processing tools. These applications rely on services provided by the operating system to function correctly.

c. Shell

The shell acts as an **interface between the user and the operating system**, receiving commands from the user and forwarding them to the system for execution. Shells may be implemented as text-based interfaces or graphical interfaces.



d. Kernel

The kernel is the **core component of the operating system** responsible for managing the CPU, memory, and hardware devices. It operates at the lowest software level and runs continuously while the system is active.

e. Hardware

Hardware forms the lowest layer of the system and consists of all physical components of the computer. All commands and processes are ultimately executed by hardware under the control of the kernel.

3. Linux File System and Shell

3.1 Linux File System Concept

In Linux, all data is stored as **files**, including documents, images, programs, and special files such as devices or sockets. Files are organized into **directories (folders)** that can contain files or subdirectories, forming a **hierarchical, tree-like structure**.

The **root directory (/)** is the starting point of the entire file system, while the **home directory (~)** serves as personal storage for each user. This separation ensures that system files are distinct from user data, keeping the system secure and organized.

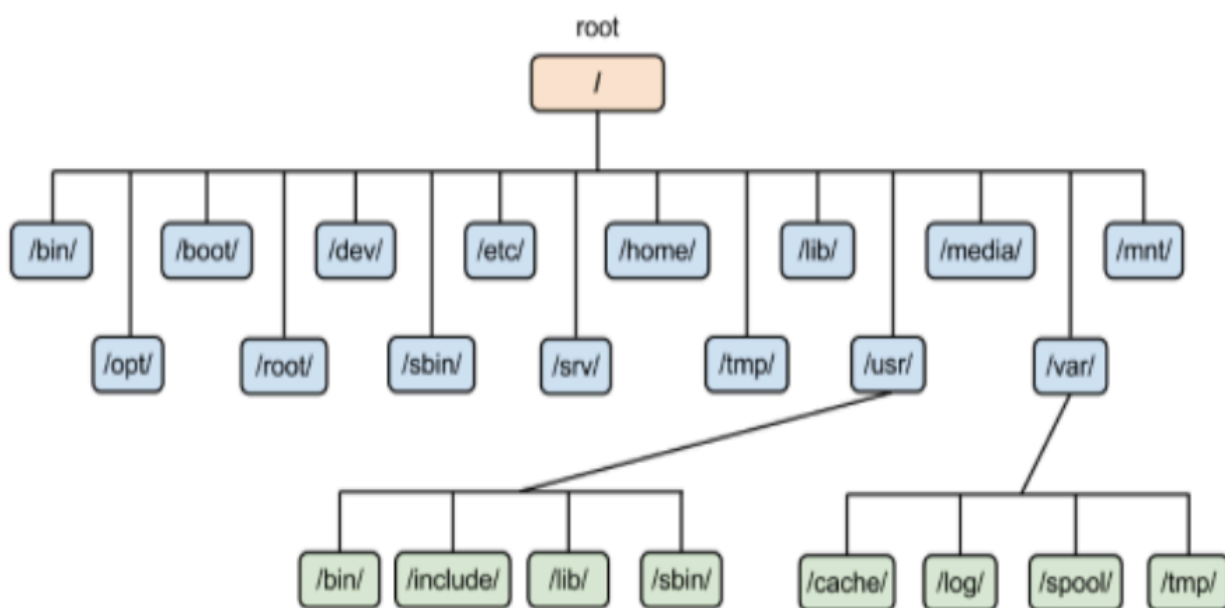


Fig 6. Simple File System Hierarchy Diagram

3.2 Basic Linux File System Hierarchy

Some important directories in Linux include **/**, **/home**, **/root**, **/etc**, **/var**, **/bin**, and **/usr**, where the **root directory (/)** is the top-level directory.





Directory	Full Name	Description
<code>/bin</code>	Binaries	Contains essential executable files (commands) available to all users. These files are marked with an "x" (executable) permission.
<code>/boot</code>	Boot Filesystem	Contains files needed to boot the system, such as the Linux Kernel (<code>vmlinuz</code>), GRUB bootloader, and boot configurations.
<code>/dev</code>	Device Filesystem	Acts as an interface to hardware. Every hardware device connected to the system is represented as a file here.
<code>/etc</code>	Etcetera	The central location for system-wide configuration files (e.g., SSH, Cron, networking, and user accounts).
<code>/home</code>	Home Directory	Stores personal data and configurations for regular users. Each user has their own subdirectory here.
<code>/lib</code>	Library Files	Contains shared library files (like <code>libc.so</code>) required by the binaries in <code>/bin</code> and <code>/sbin</code> to function.
<code>/media</code>	Removable Media	The default mount point for removable devices like USB drives, SD cards, and CD-ROMs.
<code>/mnt</code>	Mount Directory	Used by administrators to manually mount temporary filesystems, such as extra hard drives or network shares.
<code>/opt</code>	Optional Software	Reserved for "add-on" or third-party software packages that don't follow the standard file layout.
<code>/proc</code>	Process Info	A virtual filesystem providing real-time data about running processes and kernel status (e.g., <code>/proc/cpuinfo</code>).
<code>/root</code>	Root Home	The private home directory specifically for the root user (Superuser). It is separate from <code>/home</code> .
<code>/run</code>	Runtime Directory	Stores volatile runtime data for applications running since the last boot (e.g., process IDs and lock files).
<code>/sbin</code>	System Binaries	Contains essential system binaries used for administration and maintenance. Usually requires root privileges.
<code>/srv</code>	Service Data	Stores data for services provided by the system, such as web server files (<code>/srv/www</code>) or FTP data.
<code>/tmp</code>	Temporary Files	A place for temporary files. Files here are usually deleted automatically when the system reboots.
<code>/usr</code>	Unix System Resources	Contains user-installed applications, libraries, and documentation. It is the largest directory in most systems.
<code>/var</code>	Variable Data	Holds files that change size frequently, such as logs (<code>/var/log</code>), mail spools, and application caches.





This hierarchy simplifies **file organization, permission management, and separation of system and user data.**

3.3 Shell and Command Line Interface

The shell acts as an **interface between the user and the operating system**. Through the shell, users can execute programs, manage files and directories, and access system utilities. Modern Linux systems typically use **Bash** as the default shell, supporting basic commands, scripting for automation, and variables and flow control.

The **working directory** refers to the current directory where the user operates. For example:

```
praktikumso@localhost:~$ pwd
/home/praktikumso
praktikumso@localhost:~$
```

These commands display the current directory and change the working directory, respectively. Users can execute commands relative to the working directory or use absolute paths.

4. Basic File System Navigation (CLI)

4.1 Path Concept

In Linux, the location of a file or directory is determined using a **path**, which can be either **absolute** or **relative**.

Absolute path starts from the root **/** and always points to the same location.

```
praktikumso@localhost:~$ pwd
/home/praktikumso
```

Relative path specifies a file's location relative to the current working directory.

```
praktikumso@localhost:~$ ls
Desktop  Downloads  Pictures  Templates
Documents Music      Public    Videos
praktikumso@localhost:~$ cd Documents
praktikumso@localhost:~/Documents$ pwd
/home/praktikumso/Documents
praktikumso@localhost:~/Documents$
```

If the current working directory is **/home/praktikumso**, the command **cd Documents** will navigate to **/home/praktikumso/Documents**.

Special symbols often used for navigation:

'..' → parent directory. For example, move up one level





```
praktikumso@localhost:~/Documents$ pwd
/home/praktikumso/Documents
praktikumso@localhost:~/Documents$ cd ..
praktikumso@localhost:~$ pwd
/home/praktikumso
```

~ → user's home directory. For example, move to your home directory:

```
praktikumso@localhost:~/Documents$ pwd
/home/praktikumso/Documents
praktikumso@localhost:~/Documents$ cd ~
praktikumso@localhost:~$ pwd
/home/praktikumso
```

4.2 Basic Navigation Commands

Command	Function
<code>pwd</code>	Displays the current working directory
<code>ls</code>	List files and directories
<code>ls -l</code>	Lists files and directories with details (permissions, size, modification date)
<code>ls -a</code>	List all files (including hidden files)
<code>cd</code>	Changes directory
<code>mkdir</code>	Creates a new directory
<code>rmdir</code>	Removes an empty directory
<code>rm</code>	Removes a file
<code>rm -r</code>	Removes a directory along with its contents
<code>touch</code>	Creates a new empty file
<code>cp</code>	Copies a file or directory
<code>cp -r</code>	Copies a directory along with its contents
<code>mv</code>	Moves or renames a file/directory
<code>cat</code>	Displays the contents of a file
<code>head</code>	Shows the first few lines of a file
<code>tail</code>	Shows the last few lines of a file
<code>tree</code>	Displays directory structure as a tree
<code>clear</code>	Clears the terminal screen





With this table, we hope you can practice **navigating the file system, creating/deleting files & folders, viewing file contents, copying/moving files, and managing permissions**, which are essential skills for working in the Linux CLI.

EXERCISE 1 : The Traveler (Path Concept)

1. Check your current location.
2. Create a directory named **Exercise**.
3. Enter the **Exercise** directory.
4. Move up one directory level.
5. Enter the **Exercise** directory again.
6. Stay in the current directory.
7. Move up two directory levels.
8. Jump to a system directory using an absolute path.
9. Return to your home directory.

EXERCISE 2 : The Detective (Exploring Output)

Start by opening your terminal and asking the system where you are by typing `pwd`. To see what is around you, type `ls` to get a simple list of names. Now, ask for the "full story" by typing `ls -l`. Notice how the **shell** renders extra details like permissions and file sizes that were hidden before.

EXERCISE 3 : The Architect (File Management)

1. Create a new empty file named `draft.txt` using `touch`
2. Make a duplicate file named `backup.txt`.
3. Change the name to `final.txt`
4. Finally, check your work to see if there is any content inside.

EXERCISE 4 : The Cleanup (System Maintenance)

1. Delete the individual file named `final.txt`.
2. Use the command with the recursive flag to remove the entire `Exercise` directory and everything inside it.
3. Verify that the directory is gone by attempting to list the contents of your current location.

EXERCISE 5 : The Root Observer (Nested Folders & System Tree)

1. Create a folder inside another folder in one command
2. Navigate into the deepest folder and verify your path with `pwd`.
3. Now, take a look at the big picture of the Linux system by displaying the root directory's first-level structure.



4.3 Command Workflow

Every CLI command goes through several stages before producing an action on the system:

1. **Command:** the user types a command in the terminal.
2. **Shell:** the shell receives the command and checks its syntax.
3. **System Call:** the shell requests system services from the kernel according to the command.
4. **Kernel:** the kernel executes instructions and manages hardware resources.
5. **File System:** the file system performs the requested operation, such as reading a directory, creating a folder, or displaying file contents, then returns the result to the shell and displays the result to the user.

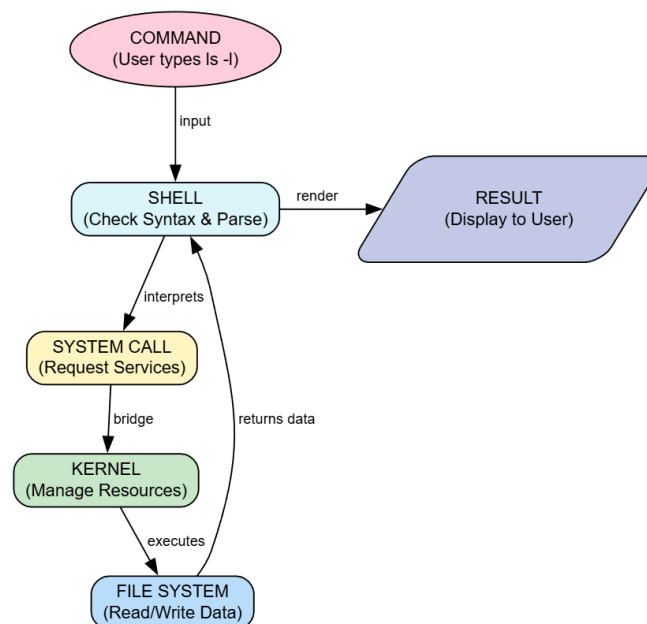


Fig 7. Command Workflow





PRACTICAL APPLICATIONS

PRACTICE 1

Scenario: You need to build a folder structure for an Operating Systems course project that separates reports from practice scripts.

- **Step 1:** Create a main folder named **ProjectOS**.
- **Step 2:** Inside **ProjectOS**, create two sub-directories at once named **Reports** and **Scripts**.
- **Step 3:** Enter the **Scripts** folder, then create an empty file named **test.sh**.
- **Step 4:** Without leaving the **Scripts** folder, use a **Relative Path** to copy the file **test.sh** into the **Reports** folder with a new name, **backup_test.txt**.
- **Step 5:** Use a special shortcut to jump directly back to your **Home Directory**.

PRACTICE 2

Scenario: You want to ensure all files are properly organized and efficiently remove unnecessary files.

- **Step 1:** Use an **Absolute Path** command to move to the **ProjectOS/Reports** folder you created earlier.
- **Step 2:** Display the contents of that folder with **full details** (including file size and creation time) to confirm that **backup_test.txt** exists.
- **Step 3:** **Rename** the file **backup_test.txt** to **final_report.txt**.
- **Step 4:** Move up one level to the **parent directory** using a special symbol.
- **Step 5:** **Delete** the entire **ProjectOS** folder along with all of its contents permanently in a single command.

DEMO - CASE STUDY

The Situation

A user is working on a Linux system and wants to create a simple project structure that separates documents, scripts, and data files. The objective is to keep project resources organized and easy to manage while working entirely from the terminal.

The Problem

The project files are not yet organized into a clear directory structure. The user needs to understand how to make a directory, make files, navigate directories, apply absolute and relative paths, and identify the current working directory in order to build the required project layout correctly.

The Mission





The user wants to create a simple project structure with folders for documents, scripts, and data. The goal is to organize project files efficiently while understanding directory navigation, absolute and relative paths, and the working directory in Linux.

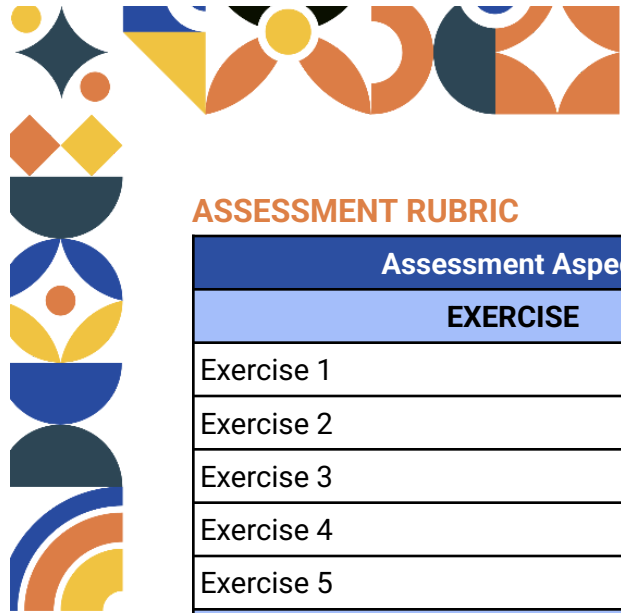
Project Structure:

```
Project/
├── Documents/
│   ├── report1.txt
│   └── report2.txt
├── Scripts/
│   ├── main.sh
│   └── helper.sh
└── Data/
    ├── data1.csv
    ├── data2.csv
    └── data3.csv
```

Demo Rules

1. **No Module or Notes Access** Students must not open the module PDF, personal notes, or online documentation during the demo session.
2. **Demonstrate Core Linux Navigation Skills** Students must clearly show proficiency in:
 - Navigating the file system
 - Efficient directory movement
 - Managing files and directories, including creating (`mkdir`, `touch`), copying (`cp`), moving (`mv`), and removing (`rm`).
 - Exploring system structures and viewing file contents
3. **Final Verification** After completing the assigned tasks, students must verify the results (e.g., confirming a file has been moved or a directory has been deleted) to ensure the system state matches the mission requirements.
4. **Live Conceptual Assessment** During or after the demo, the assistants will ask several **key questions** to evaluate your **Conceptual Understanding**.
Note: Please prepare thoroughly, as this accounts for a significant portion of your grade.
5. **Follow Live Instructions** All actions must be performed interactively following the instructor's or assistant's directions.

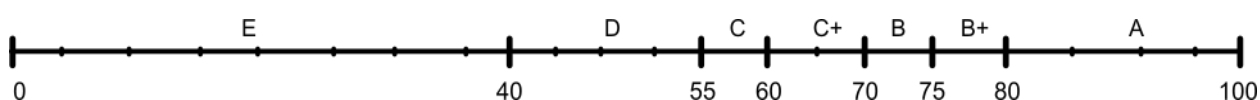




ASSESSMENT RUBRIC

Assessment Aspect	Points (Total 100%)
EXERCISE	Total 5%
Exercise 1	1%
Exercise 2	1%
Exercise 3	1%
Exercise 4	1%
Exercise 5	1%
PRACTICE	Total 20%
Practice 1 (Verbal assessment)	5%
Practice 2 (Verbal assessment)	5%
Practice Completion (Live verification)	10%
DEMO	Total 75%
Case Study Completion	35%
Understanding Directory Manipulation Logic	15%
Understanding Navigation Logic Mastery	10%
Output Validation	10%
Troubleshooting	5%

ASSESSMENT SCALE



- A** = (81 - 100) → Sepuh
- B+** = (75 - 80) → Very Good
- B** = (70 - 74) → Good
- C+** = (60 - 69) → Fairly Good
- C** = (55 - 59) → Fair
- D** = (41 - 54) → Poor
- E** = (0 - 40) → Bro really...

